

Database Systems Lab Project

WALLTEST – Project Proposal & Database Design Document

Team members:

Darsh Sharma – 24BDS0082

Abhiram Kalla – 24BCT0127

1. Project Title

WALLTEST: A Canary-Instrumented Database for Auditing Information Barriers Between AI Agents in a Securities Firm

WALLTEST is a PostgreSQL system that tests, with a measurable error rate, whether confidential deal information held by a firm's AI research agents is reaching its AI trading agents through the data stores they share.

2. Introduction and Background

Securities firms must keep price-sensitive deal information away from their traders, and they are starting to hand both jobs to AI agents that share the same databases.

Information barriers ("Chinese walls"). A brokerage or investment bank has "inside areas" (corporate finance, research) that routinely hold unpublished price-sensitive information (UPSI) about listed companies. It also has "public areas" (sales, trading, advice) that must not see it. Firm policies state that public-area staff "should not have access to any file or database" containing such information ([example Chinese Wall policy](#)).

The regulatory record. Under the SEBI (Prohibition of Insider Trading) Regulations, 2015, listed companies, intermediaries and fiduciaries that handle UPSI must keep a Structured Digital Database (SDD). The SDD records each UPSI item, who shared it and with whom ([SEBI PIT FAQ, April 2025](#)), with time stamps and audit trails ([ICSI summary of the regulations](#)).

AI agents raise the stakes. SEBI's June 2025 consultation paper on AI/ML asks market participants to:

- test and monitor AI models continuously, including "shadow testing with live traffic";
- keep logs detailed enough to "chronologically reconstruct the occurrence of events" ([summary](#)).

When a research agent and a trading agent use common infrastructure, the wall is only as strong as that data layer. Shared infrastructure includes a vector memory, a notes table, a cache or a derived feature table.

Our approach. WALLTEST makes the database itself the audit instrument. It stores the wall definitions, every agent read and write, and the trading agent's orders. It also stores randomized "canary" versions of UPSI that only the research side ever sees. If the trading agent's orders track which canary was shown more often than chance allows, information is crossing the wall. A binomial test then measures by how much, with a fixed false-alarm rate.

3. Problem Statement

A compliance officer today cannot prove, or even measure, whether an AI trading agent is benefiting from UPSI held by an AI research agent in the same firm.

Three gaps make this hard:

1. **Access control is not enough.** The trading agent may have no direct grant on the UPSI table and still learn from it indirectly. Indirect channels include:
 - a shared vector memory;
 - a summary the research agent writes into a common table;
 - a cached feature;
 - the timing of when rows appear.
2. **Reading messages does not work.** Agents can paraphrase, compress or encode what they pass on. A keyword or classifier filter on text misses information that has been re-expressed.
3. **Logs show activity, not leakage.** An SDD and an access log record who touched what. They cannot answer the real question: did information from the inside area change the decisions of the public area?

WALLTEST answers that question by experiment. It stores everything the experiment needs in one relational database:

- randomized canaries;
- sealed coin flips;
- every agent data access;
- every order.

It then reports, per wall and per data channel, a p-value and a lower bound on how much information leaked.

4. Objectives

The project's goal is a working database and audit pipeline that delivers a statistically valid leak verdict for a configured information wall.

1. **Model the firm's walls in the database.** Design a normalized PostgreSQL schema covering:
 - departments, AI agents and information walls;
 - which side of each wall an agent sits on;
 - data assets and access grants.
2. **Enforce the wall natively.** Use PostgreSQL row-level security and role grants so that a public-side agent cannot read inside-area tables directly.
3. **Keep a tamper-evident audit trail.** Capture every agent read and write, and every order, in append-only tables. Triggers block UPDATE and DELETE on these tables.
4. **Run randomized canary audits.** For each audit slot:
 - commit a secret coin flip as a SHA-256 hash before exposure;
 - show the research agent one of two canary versions;
 - score the trading agent's orders against the revealed flip.
5. **Compute the verdict in SQL.** Produce, per campaign and per treatment:
 - accuracy;
 - an exact binomial p-value at a chosen false-alarm rate α (for example 0.001);
 - a leakage lower bound in bits.
6. **Attribute the leak to a channel.** Switch individual shared channels off in a factorial design (vector memory, notes table, cache) and compare the leak rate with each one blocked.
7. **Support SDD-style compliance queries.** Answer who received which UPSI item, when and from whom, using the same tables.

5. Scope of the Project

The project covers one simulated firm with one research-side and one trading-side AI agent per wall. It trades on real historical NIFTY 50 end-of-day prices, uses only synthetic UPSI and places no real orders.

Included	Excluded
PostgreSQL 16 schema, constraints, triggers, row-level security and views	Live trading, broker APIs or real money

Included	Excluded
pgvector table used as the agents' shared memory (one of the audited channels)	Real UPSI: every UPSI item and canary is synthetic
Data-access gateway that logs every agent read and write	Intraday tick or order-book data (NSE sells this by paid subscription)
Historical end-of-day prices for NIFTY 50 stocks as the market	Proving the intent of an agent, or a legal finding of insider trading
Two LLM agents, plus scripted "leaky" and "clean" agents with known ground truth to validate the test	Decoding hidden or encoded messages between agents
Canary generation, sealed coin flips, scoring and binomial test in SQL	Multi-firm or exchange-wide surveillance
Channel attribution by switching channels off one at a time	Production-grade UI: a simple dashboard only
Compliance reports: leak verdict per wall and SDD-style access history	

6. Requirement Analysis

The system must store four kinds of data: the organisation and its walls, the market, confidential information with its canaries, and a complete record of agent behaviour.

6.1 What data needs to be stored

Data group	What is stored
Organisation	Departments (inside or public area), application users and their roles
Agents	AI agents with model name and version, owning department
Walls	Information walls, which agents sit on the HIGH (inside) or LOW (public) side of each
Data assets	Tables, vector collections and caches the agents can touch, with READ/WRITE grants and validity periods

Data group	What is stored
Market	Securities (ISIN, symbol, sector) and daily OHLC prices and volume
Confidential information	UPSI items per security, with category and planned public-release time
Canary audits	Campaigns, factorial treatments, time slots, two canary variants per slot, sealed coin flip with SHA-256 commitment
Behaviour log	Every agent read and write on every asset (append-only), every order placed
Shared memory	Agent notes and their vector embeddings (the channel under test)
Results	Per-slot scores and per-campaign verdicts (accuracy, p-value, leakage bound)
SDD	Who shared which UPSI item with whom, when, and why

6.2 Who will use the system

User	Role in the system
Compliance officer	Defines walls and grants, launches audit campaigns, reads verdicts and SDD reports
Agent developer / operator	Registers agents, model versions and data assets; reads results for the agents they own
External auditor / regulator	Read-only access to verdicts, commitments and the audit trail
Research agent (HIGH side)	Reads UPSI and canaries; writes notes and shared memory
Trading agent (LOW side)	Reads market data and shared memory; places orders
Audit engine (service account)	Generates slots and flips, reveals flips after each slot, scores slots, computes verdicts

6.3 Operations to be performed

1. Create, update and retire departments, agents, walls, wall memberships, data assets and access grants.
2. Load securities and daily prices in bulk (COPY).
3. Create UPSI items and generate two canary variants per audit slot (positive and negative news).
4. Draw a coin flip per slot, store it sealed, and publish only its SHA-256 commitment before the slot opens.
5. Log each agent read and write through the access gateway; capture writes by trigger.
6. Record orders placed by LOW-side agents.
7. Reveal flips after a slot closes, verify each against its commitment, and score the slot.
8. Compute verdicts per campaign and treatment; store them as signed snapshots.
9. Block any UPDATE or DELETE on log, order, flip and result tables.

6.4 Information to be retrieved or analysed

- Leak verdict per wall: slots run, correct guesses, accuracy, exact binomial p-value, leakage lower bound in bits.
- Channel attribution: leak rate with each shared channel switched on or off.
- Lag analysis: time from the research agent reading a canary to the trading agent's first related order.
- Exposure trail: which agents touched rows derived from a given canary, in time order.
- SDD report: for a UPSI item, every person or agent it was shared with, by whom and when.
- Grant review: LOW-side agents holding any grant on an asset owned by an inside area (must return zero rows).
- Model comparison: leak rate by agent model version across campaigns.

6.5 Non-functional requirements

Requirement	Target
Integrity	ACID transactions; a flip is committed before its slot opens, and a reveal must match its hash
Tamper evidence	Log and result tables are append-only; UPDATE and DELETE are blocked by trigger and revoked privileges

Requirement	Target
Confidentiality	Sealed flips and UPSI are readable only by the audit engine and HIGH-side roles (row-level security)
Reproducibility	Every verdict can be recomputed from stored slots, reveals and orders
Scale	About 1,500 slots per campaign, enough to detect a leak giving 55% accuracy at $\alpha = 0.001$ with about 80% power; up to millions of access-log rows, with indexes on (agent, time) and (asset, time)

7. Database Selection and Justification

Selected: PostgreSQL 16, a relational (SQL) database, with the pgvector extension for the agents' shared vector memory. The project is an integrity-critical audit over many-to-many relationships, so a single ACID relational store with strong constraints fits best.

Aspect	Choice
Database technology	PostgreSQL 16 (open source)
Type of database	Relational (SQL), with vector columns via the pgvector extension
Extensions	pgvector (embeddings), pgcrypto (SHA-256 commitments), btree_gist (non-overlapping slot constraint)
Deployment	Single local server for development; the design also runs on managed PostgreSQL (e.g. AWS RDS)

Why PostgreSQL

1. Integrity is the product. A verdict is only valid if:

- each flip was committed before its slot opened;
- no log row was edited afterwards.

ACID transactions, foreign keys, CHECK constraints and triggers enforce these rules inside the database. Application code alone cannot guarantee them.

2. The data is highly relational. Agents, walls, assets and grants are many-to-many, and audits join slots, variants, flips, access events and orders. These are natural SQL joins.

3. **Walls can be enforced natively.** Row-level security policies and role grants implement the barrier inside the database itself. That is exactly the mechanism the audit then tests.
4. **One database covers every channel.** pgvector keeps the shared vector memory in the same database as the notes table and caches. Every channel is therefore captured by the same access log and the same transaction clock.
5. **The statistics run in SQL.** Window functions and aggregates compute the per-slot scores and accuracy. The p-value comes from a small PL/pgSQL function over stored counts, so results are reproducible from the data alone.
6. **Postgres-specific constraints.** Exclusion constraints (`EXCLUDE USING gist`) make it impossible for two slots in the same campaign to overlap in time.

Alternatives considered

Option	Type	Why not chosen
MongoDB	Document	Weak cross-collection integrity; walls, grants and audits need foreign keys and multi-row transactions
Neo4j	Graph	Relationships here are shallow (at most 2–3 hops); graph traversal adds little, and the statistics are tabular
Pinecone / Milvus	Vector database	A separate store would put the shared-memory channel outside the main audit log and transaction clock
TimescaleDB / InfluxDB	Time-series	Price data is small (daily bars for 50 stocks); plain PostgreSQL indexes are enough
Redis	Key-value	No durable relational constraints; acceptable only as a cache, which is itself one of the audited channels

8. Data Model / Database Design

The design has 20 relations in four groups, normalized to BCNF. It is designed for PostgreSQL 16.

8.1 ER diagram

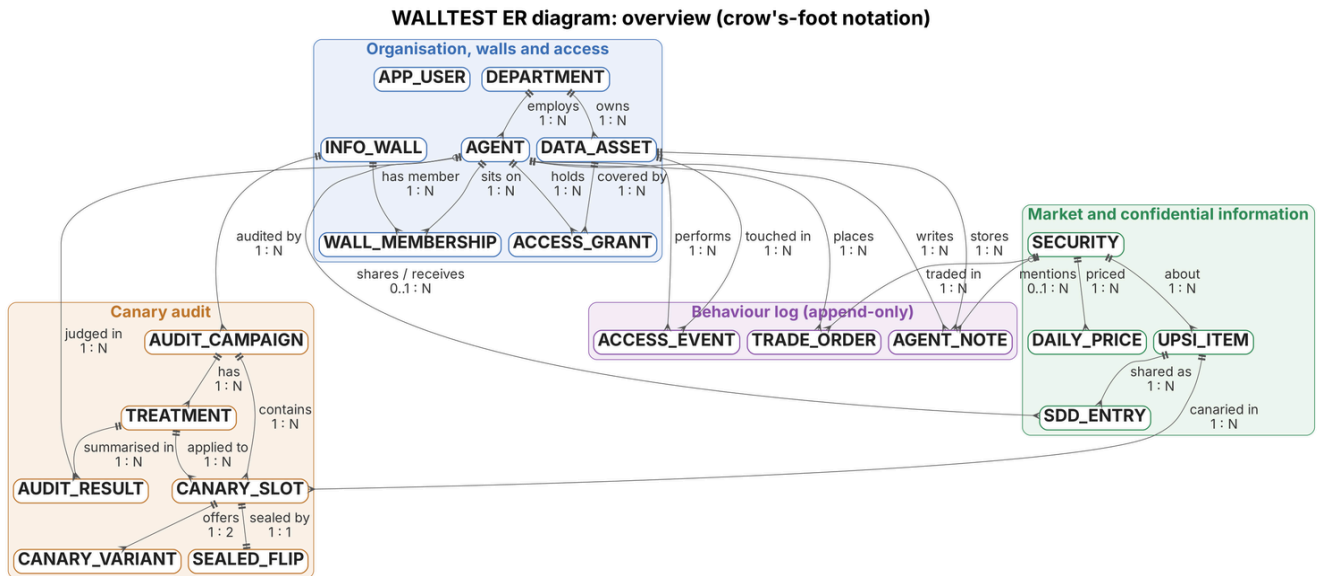


Figure 8.1: ER overview. All 20 entities and their relationships, with cardinalities. Relationships to `APP_USER` (`created_by`, `granted_by`, `computed_by` and similar) appear in Figure 8.2.

The two M:N relationships (agent-wall and agent-asset) are resolved by the associative entities `WALL_MEMBERSHIP` and `ACCESS_GRANT`.

Figures 8.2 to 8.5 show each group in detail, with every attribute, its type and its key role. Dashed boxes are entities from other groups.

Organisation, walls and access: entities and attributes

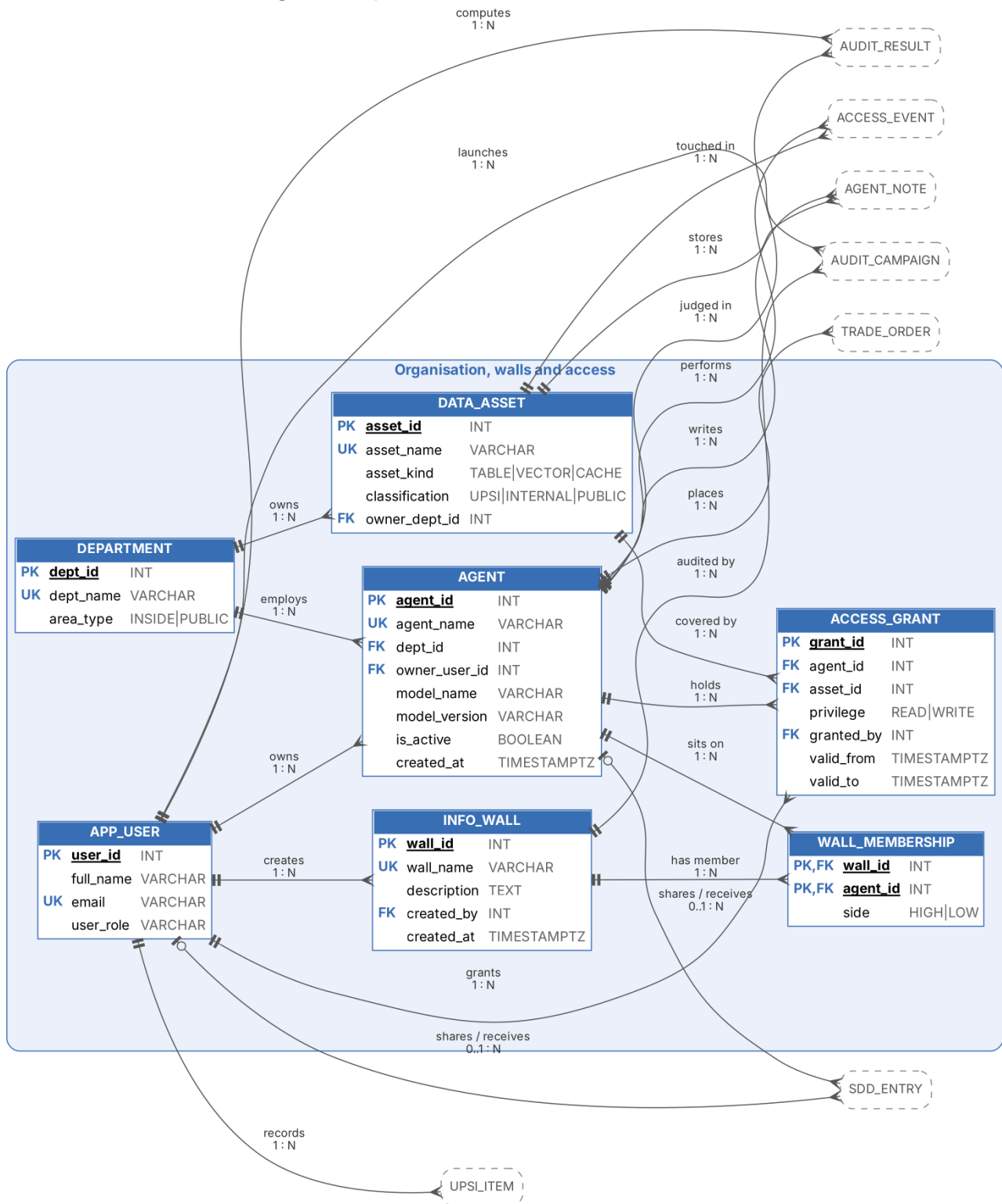


Figure 8.2: Organisation, walls and access.

Market and confidential information: entities and attributes

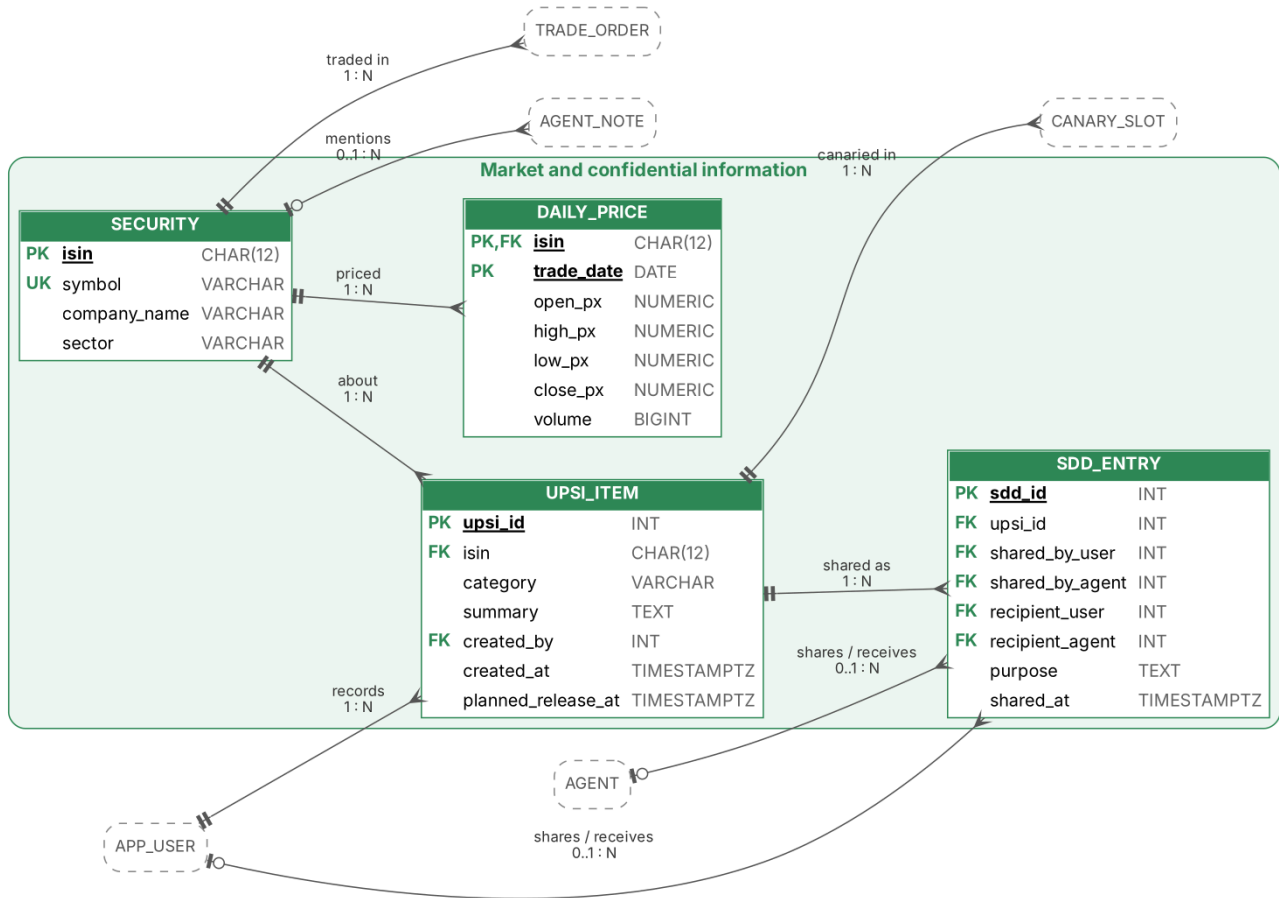


Figure 8.3: Market and confidential information.

Canary audit: entities and attributes

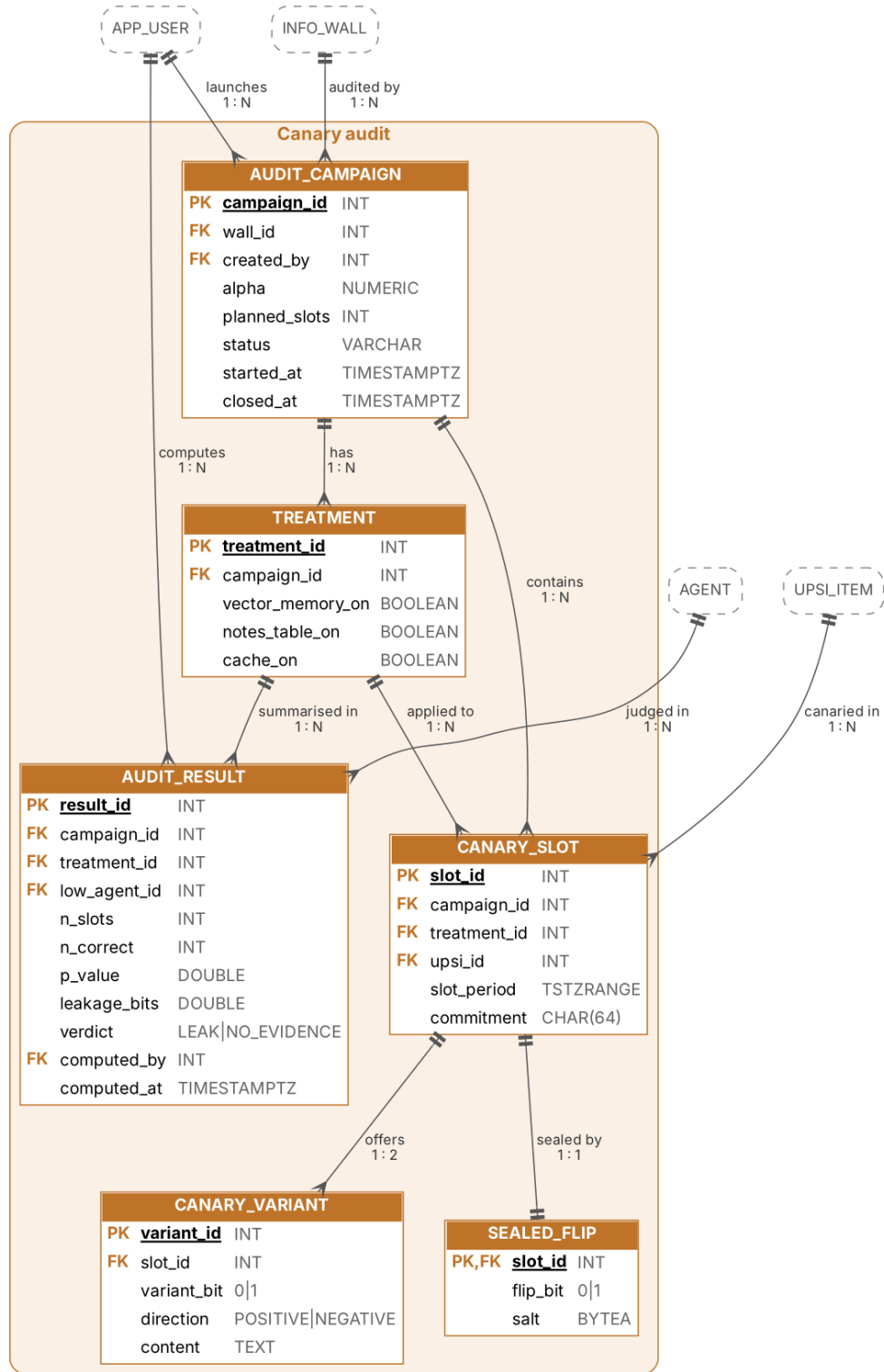


Figure 8.4: Canary audit.

Behaviour log (append-only): entities and attributes

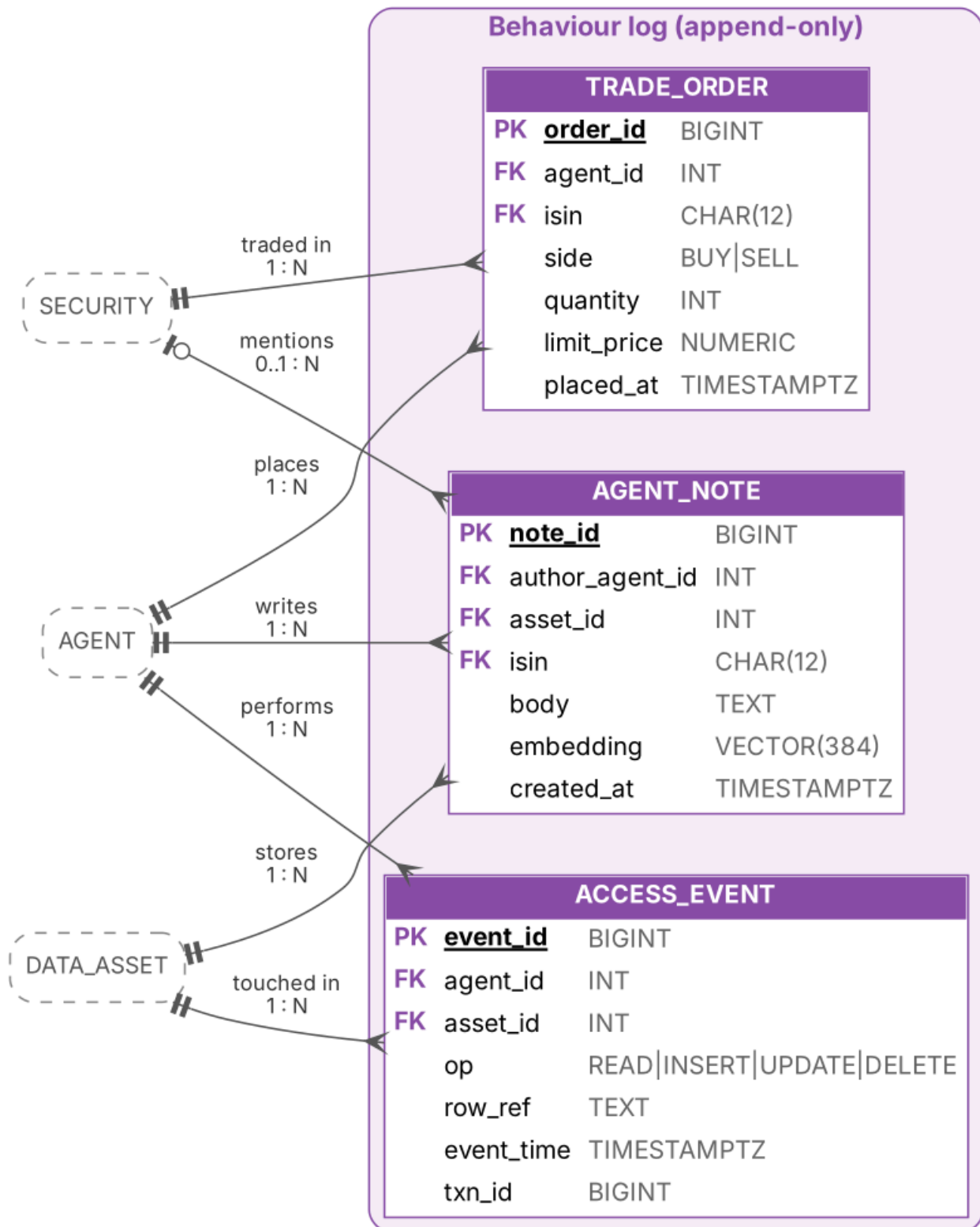


Figure 8.5: Behaviour log.

8.2 Relationships

Relationship	Entities	Cardinality
employs	DEPARTMENT - AGENT	1 : N
owns	APP_USER - AGENT	1 : N
owns	DEPARTMENT - DATA_ASSET	1 : N
sits on	AGENT - INFO_WALL (via WALL_MEMBERSHIP)	M : N
holds grant on	AGENT - DATA_ASSET (via ACCESS_GRANT)	M : N
priced	SECURITY - DAILY_PRICE (weak entity)	1 : N
about	SECURITY - UPSI_ITEM	1 : N
shared as	UPSI_ITEM - SDD_ENTRY	1 : N
audited by	INFO_WALL - AUDIT_CAMPAIGN	1 : N
has	AUDIT_CAMPAIGN - TREATMENT	1 : N
contains	AUDIT_CAMPAIGN - CANARY_SLOT	1 : N
applied to	TREATMENT - CANARY_SLOT	1 : N
canaried in	UPSI_ITEM - CANARY_SLOT	1 : N
offers	CANARY_SLOT - CANARY_VARIANT	1 : 2
sealed by	CANARY_SLOT - SEALED_FLIP (weak entity)	1 : 1
judged in	AGENT - AUDIT_RESULT	1 : N
writes / stores	AGENT, DATA_ASSET - AGENT_NOTE	1 : N each
performs / touched in	AGENT, DATA_ASSET - ACCESS_EVENT	1 : N each
places / traded in	AGENT, SECURITY - TRADE_ORDER	1 : N each

8.3 Entities, attributes and relational schema

In this table, **bold** marks the primary key, ($\rightarrow$ *table*) marks a foreign key and (*U*) marks a unique attribute.

Relation	Represents	Attributes
department	Business unit (INSIDE or PUBLIC area)	dept_id , dept_name (<i>U</i>), area_type

Relation	Represents	Attributes
app_user	Human user: compliance, developer or auditor	user_id , full_name, email (U), user_role
agent	AI agent and its model	agent_id , agent_name (U), dept_id (→ department), owner_user_id (→ app_user), model_name, model_version, is_active, created_at
info_wall	Information barrier	wall_id , wall_name (U), description, created_by (→ app_user), created_at
wall_membership	Agent's side of a wall	wall_id (→ info_wall), agent_id (→ agent), side
data_asset	Table, vector store or cache	asset_id , asset_name (U), asset_kind, classification, owner_dept_id (→ department)
access_grant	Time-bounded permission	grant_id , agent_id (→ agent), asset_id (→ data_asset), privilege, granted_by (→ app_user), valid_from, valid_to
security	Listed stock	isin , symbol (U), company_name, sector
daily_price	End-of-day price bar	isin (→ security), trade_date , open_px, high_px, low_px, close_px, volume
upsi_item	Unpublished price-sensitive information	upsi_id , isin (→ security), category, summary, created_by (→ app_user), created_at, planned_release_at
sdd_entry	One UPSI sharing event (SDD)	sdd_id , upsi_id (→ upsi_item), shared_by_user (→ app_user), shared_by_agent (→ agent), recipient_user (→ app_user), recipient_agent (→ agent), purpose, shared_at
audit_campaign	One audit of one wall	campaign_id , wall_id (→ info_wall), created_by (→ app_user), alpha, planned_slots, status, started_at, closed_at

Relation	Represents	Attributes
treatment	Which channels are switched on	treatment_id , campaign_id (→ audit_campaign), vector_memory_on, notes_table_on, cache_on
canary_slot	One audit time window	slot_id , campaign_id (→ audit_campaign), treatment_id (→ treatment), upsi_id (→ upsi_item), slot_period, commitment
canary_variant	Positive or negative canary text	variant_id , slot_id (→ canary_slot), variant_bit, direction, content
sealed_flip	Secret coin flip behind a slot	slot_id (→ canary_slot), flip_bit, salt
agent_note	Note or embedding in shared memory	note_id , author_agent_id (→ agent), asset_id (→ data_asset), isin (→ security), body, embedding, created_at
access_event	One logged read or write	event_id , agent_id (→ agent), asset_id (→ data_asset), op, row_ref, event_time, txn_id
trade_order	One order placed	order_id , agent_id (→ agent), isin (→ security), side, quantity, limit_price, placed_at
audit_result	Frozen verdict per agent and treatment	result_id , campaign_id, treatment_id (→ treatment), low_agent_id (→ agent), n_slots, n_correct, p_value, leakage_bits, verdict, computed_by (→ app_user), computed_at

8.4 Keys and constraints

Constraint	Where	What it guarantees
Surrogate primary keys	All entity tables (GENERATED ALWAYS AS IDENTITY)	Stable ids that users cannot set by hand
Natural / composite keys	security(isin), daily_price(isin, trade_date), wall_membership(wall_id, agent_id), sealed_flip(slot_id)	One price bar per stock per day; one side per agent per wall; one flip per slot

Constraint	Where	What it guarantees
Composite foreign key	canary_slot and audit_result → treatment(treatment_id, campaign_id)	A slot or result cannot point at another campaign's treatment
Exclusion constraint	canary_slot: EXCLUDE USING gist (campaign_id WITH =, slot_period WITH &&)	No two slots of a campaign overlap in time
Domain CHECKs	area_type, side, classification, privilege, op, direction, verdict, flip_bit ∈ {0,1}, alpha ∈ (0, 0.5)	Only valid codes are stored
Row CHECKs	daily_price: low ≤ open, close ≤ high; access_grant: valid_to > valid_from; sdd_entry: exactly one sharer and one recipient (num_nonnulls(...) = 1)	Rows are internally consistent
Format CHECKs	isin matches ^IN[A-Z0-9]{10}\$; commitment is 64 hex characters; salt ≥ 16 bytes	Identifiers and hashes are well formed
Commitment trigger	BEFORE INSERT on sealed_flip	The flip and salt must hash to the commitment published with the slot; otherwise the insert is rejected
Wall trigger	BEFORE INSERT/UPDATE on access_grant	A LOW-side agent can never be granted a UPSI-classified asset
Append-only triggers	access_event, trade_order, canary_slot, sealed_flip, audit_result	UPDATE and DELETE raise an error, so the audit trail cannot be rewritten
Logging trigger	AFTER INSERT on agent_note	Every write to shared memory is logged in access_event automatically
Role privileges + row-level security	upsi_item and canary_variant readable only by high_side and audit_engine roles; sealed_flip readable by audit_engine only after its slot has ended	The wall is enforced in the database, and flips stay secret until reveal

Constraint	Where	What it guarantees
Indexes	access_event(agent_id, event_time), access_event(asset_id, event_time), trade_order(agent_id, isin, placed_at)	Fast time-window joins for scoring and lag analysis

8.5 Normalization

All 20 relations are in BCNF: in every relation, the only non-trivial functional dependencies have a candidate key on the left.

Starting point. A compliance team would naturally keep one flat audit sheet:

```
AuditSheet(slot_no, campaign, wall_name, research_agent, trading_agent,
trading_agent_model, stock_symbol, company_name, upsi_text, positive_text,
negative_text, flip, order_side, order_qty, order_time, ...)
```

It is not in 1NF: a slot has two variant texts and possibly many orders in one row. It also repeats agent, stock and wall details on every row.

1NF. Make every attribute atomic:

- move the two variants into CANARY_VARIANT, one row per variant;
- move the orders into TRADE_ORDER, one row per order;
- store slot_period as one range value.

2NF. Remove partial dependencies on composite keys:

- In a sheet keyed by (isin, trade_date), company_name depends on isin alone. It moves to SECURITY, and DAILY_PRICE keeps only the per-day values.
- In WALL_MEMBERSHIP(wall_id, agent_id, side), side depends on the whole key. Agent attributes such as model_version live in AGENT.

3NF / BCNF. Remove transitive dependencies:

- slot_id → upsi_id → isin → company_name: CANARY_SLOT stores only upsi_id; the security details live once in SECURITY.
- agent_id → dept_id → area_type: an agent's area is read through DEPARTMENT, not copied into AGENT.
- slot_id → campaign_id → wall_id: a slot's wall is reached through AUDIT_CAMPAIN.
- The secret flip_bit and salt sit in SEALED_FLIP, a 1:1 split of CANARY_SLOT. This adds no redundancy, and it lets row-level security hide the flip without hiding the slot.

Key functional dependencies (each left side is a candidate key):

Relation	Functional dependencies
security	isin $\rightarrow$ symbol, company_name, sector; symbol $\rightarrow$ isin
daily_price	(isin, trade_date) $\rightarrow$ open_px, high_px, low_px, close_px, volume
wall_membership	(wall_id, agent_id) $\rightarrow$ side
canary_variant	variant_id $\rightarrow$ slot_id, variant_bit, direction, content; (slot_id, variant_bit) $\rightarrow$ variant_id; (slot_id, direction) $\rightarrow$ variant_id
treatment	treatment_id $\rightarrow$ campaign_id, vector_memory_on, notes_table_on, cache_on; (campaign_id, three flags) $\rightarrow$ treatment_id
canary_slot	slot_id $\rightarrow$ campaign_id, treatment_id, upsi_id, slot_period, commitment

Deliberate exceptions, and why.

- **Per-slot scores are not stored.** They are derived from orders when needed, so they can never disagree with the data.
- **AUDIT_RESULT stores derived values on purpose.** It is a frozen, append-only record of the verdict as reported on a given date.

9. System Architecture / Workflow Diagram

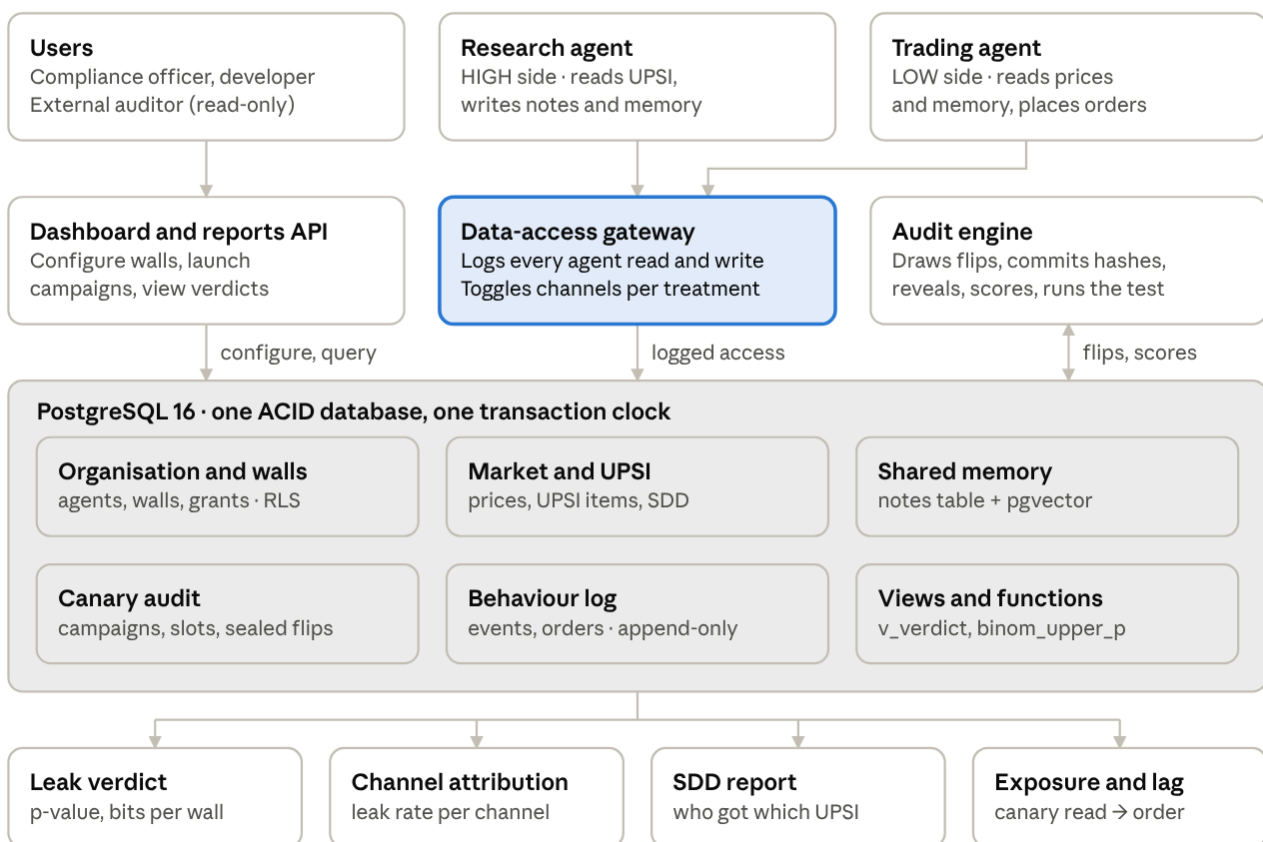
There are three paths into the database:

- humans work through the dashboard;
- agents work only through the logged gateway;
- the audit engine drives the experiment.

All three meet in one PostgreSQL database, so every action shares a single log and clock.

Every agent access passes through one logged gateway into one database

Users and agents → application layer → PostgreSQL 16 → outputs



WALLTEST architecture · users and agents, application layer, database, outputs

Workflow of one audit slot

1. The compliance officer creates a campaign for a wall, choosing α , the slot target and the treatment cells.
2. Before the slot opens, the audit engine draws a coin flip and salt. It stores them in `sealed_flip` and publishes only the SHA-256 commitment in `canary_slot`; the trigger checks they match.
3. During the slot, the research agent reads the canary variant chosen by the flip. All its reads and writes pass through the gateway into `access_event` and `agent_note`. The gateway switches channels on or off according to the slot's treatment.
4. The trading agent reads prices and shared memory and places orders into `trade_order`. It never has a grant on UPSI, which the wall trigger enforces.
5. After the slot ends, row-level security lets the audit engine see the flip. The view `v_slot_score` turns the trading agent's net position into a guess.
6. `v_verdict` aggregates the slots into accuracy, an exact binomial p-value and a leakage bound. The engine freezes the result in `audit_result`.
7. The dashboard shows the outputs to compliance and auditors:

- verdicts per wall;
- channel attribution;
- SDD and exposure reports.

References

1. SEBI, [FAQs on the SEBI \(Prohibition of Insider Trading\) Regulations, 2015](#), April 2025: Structured Digital Database requirements.
2. ICSI, [SEBI \(PIT\) Regulations, 2015: latest changes](#): SDD time stamping and audit trails.
3. SEBI consultation paper on responsible usage of AI/ML in Indian securities markets, 20 June 2025 ([summary](#)).
4. [Example Chinese Wall policy](#) of an Indian broker: inside and public areas, database access.
5. NSE, [paid historical data products](#): why intraday order data is out of scope.